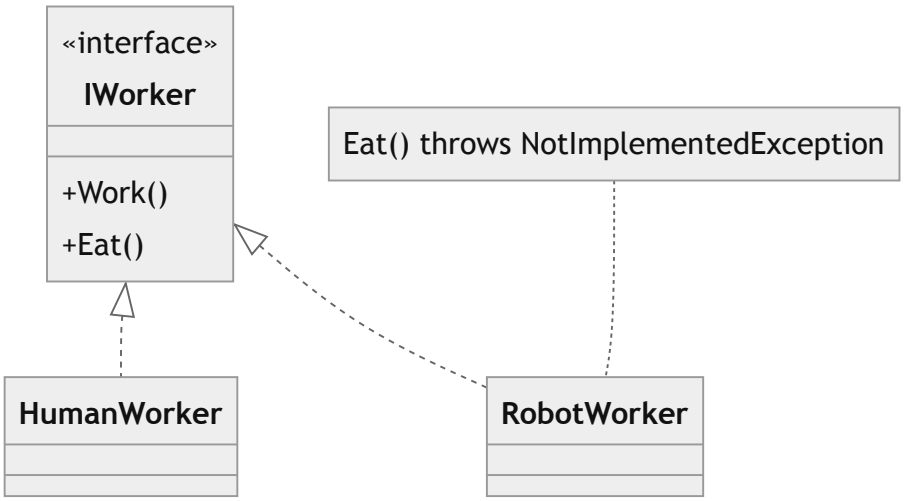
MCQ Practice

Choose the best answer for each question. Name: _____

Q1

In the design below, `RobotWorker` cannot eat, so its `Eat()` throws `NotImplementedException`. Which SOLID principle does the design of the `IWorker` interface violate?



a. Single Responsibility Principle.
b. Open-Closed Principle.
c. Dependency Inversion Principle.
d. Liskov Substitution Principle.
e. Interface Segregation Principle.

Q2

Methods in a class are grouped because they must be called in a certain order (e.g. `CheckPassword()` then `LoadMenu()`), even though they work on different data. This is an example of:

a. Procedural cohesion
b. Sequential cohesion
c. Functional cohesion
d. Logical cohesion
e. Communicational cohesion

Q3

Tests that are written only from the specification and contract of a method, without looking at its source code, are called:

a. Glass box tests
b. Black box tests
c. Integration tests
d. Performance tests
e. Security tests

Q4

What does the code below print?

```
void AddTen(int x) {
    x += 10;
}

void AddTenRef(ref int x) {
    x += 10;
}

int a = 5;
int b = 5;
AddTen(a);
AddTenRef(ref b);
Console.Write(a + " " + b);
```

a. `5 5`
b. `15 15`
c. `15 5`
d. `5 15`
e. Compile-time error.

Q5

Consider these statements about **overloaded** methods in C#:

I. They must have the same identifier.
II. They must have different parameter lists.
III. They can differ only by their return type.

Which statements are correct?

a. I only
b. I and II only
c. II and III only
d. I, II and III
e. None of the above

Q6

What does the code below do?

```
class Traveller {
    public virtual string GetTier() {
        return "Standard";
    }
}

class GoldTraveller : Traveller {
    public override string GetTier() {
        return "Gold";
    }
}

List<Traveller> travellers = new List<Traveller>();
travellers.Add(new Traveller());
travellers.Add(new GoldTraveller());

foreach (Traveller t in travellers) {
    Console.WriteLine(t.GetTier());
}
```

a. Compile-time error, because a `List<Traveller>` cannot store a `GoldTraveller`.
b. Prints `Standard`, then `Standard`, because the list only knows about `Traveller`.
c. Prints `Standard`, then `Gold`.
d. Throws `InvalidCastException` at run time.
e. Prints `Gold`, then `Gold`.

Q7

A developer reads the source code of `IncomeTax`, notices a special `if` branch for an income of exactly 18200, and writes a test for that case. This is an example of:

a. Glass box testing
b. Black box testing
c. Acceptance testing
d. Regression testing
e. System testing

Q8

In C#, which statement about `ref` and `out` parameters is correct?

a. `ref` and `out` behave exactly the same.
b. A `ref` variable must be assigned before it is passed in; an `out` parameter must be assigned inside the method before it returns.
c. An `out` variable must be assigned before it is passed in; a `ref` parameter must be assigned inside the method.
d. Both only work with reference types such as classes.
e. Both pass a copy of the value, so the caller never sees changes.

Q9

`Print(report, true)` prints a PDF and `Print(report, false)` prints plain text. The caller passes a flag that decides what the method does. Which type of coupling is this?

a. Data coupling
b. Stamp coupling
c. Control coupling
d. Content coupling
e. Common coupling

Q10

`Penguin` inherits from `Bird`, and `Penguin.Fly()` throws an exception. Which redesign best fixes the problem?

a. Wrap every call to `Fly()` in a try-catch.
b. Move `Fly()` into a separate `FlyingBird` class (or `IFlyable` interface) that only flying birds use.
c. Make `Penguin.Fly()` do nothing and return silently.
d. Make `Bird` a sealed class.
e. Check `if (bird is Penguin)` before every call to `Fly()`.

Q11

What does the code below print?

```
class A {
    public void Show() {
        Console.Write("A");
    }
}

class B : A {
    public new void Show() {
        Console.Write("B");
    }
}

A obj = new B();
obj.Show();
```

a. A
b. B
c. AB
d. BA
e. Compile-time error.

Q12

After fixing a bug, a developer re-runs all the existing tests to make sure everything that worked before the change still works. This is:

a. Acceptance testing
b. Performance testing
c. Security testing
d. Integration testing
e. Regression testing

Q13

In the code below, `Print()` crashes with a `NullReferenceException` if `Load()` has not been called first. Which type of coupling is this?

```
class SalesReport {
    private List<Sale> sales;

    public void Load(List<Sale> sales) {
        this.sales = sales;
    }
    public void Print() {
        foreach (Sale sale in sales) { /* ... */ }
    }
}

SalesReport report = new SalesReport();
report.Print();
```

a. Content coupling
b. Common coupling
c. Control coupling
d. Stamp coupling
e. Temporal coupling

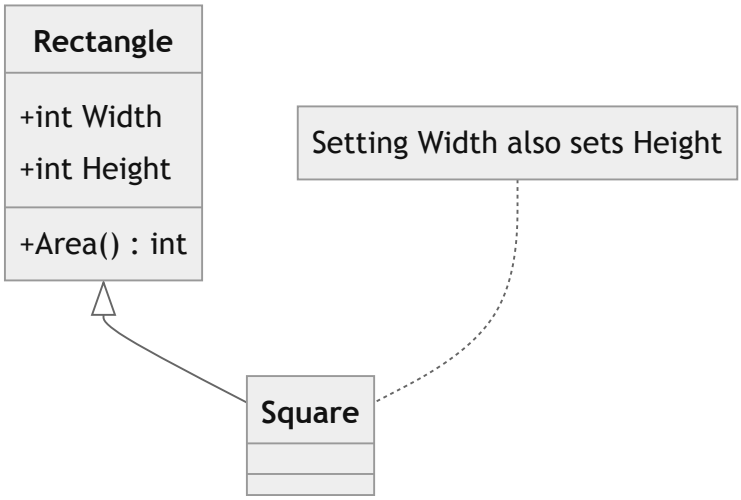
Q14

What is the difference between **overloading** and **overriding** a method in C#?

a. They are the same thing; both just mean writing a method with the same name twice.
b. Overloading needs `virtual` and `override`; overriding needs a different parameter list.
c. Overloading changes only the return type; overriding changes only the method name.
d. Overloading only works across a parent and child class; overriding only works inside one class.
e. Overloading means the same method name with a different parameter list; overriding means a child class replaces a parent's `virtual` method, keeping the same signature.

Q15

In the design below, setting `Width` on a `Square` also changes its `Height`. A method that takes a `Rectangle`, sets `Width = 5` and `Height = 4`, then expects `Area() == 20`, fails when given a `Square`. Which SOLID principle is violated?



a. Single Responsibility Principle.
b. Open-Closed Principle.
c. Liskov Substitution Principle.
d. Interface Segregation Principle.
e. Dependency Inversion Principle.

Q16

`SendWelcomeEmail(Traveller traveller)` only uses `traveller.Email`, but receives the whole `Traveller` object. Which type of coupling is this?

a. Stamp coupling
b. Data coupling
c. Control coupling
d. Common coupling
e. Content coupling

Q17

Consider these statements about **overriding** a method in C#:

I. The child method has the same identifier as the parent method.
II. The child method has the same parameter list as the parent method.
III. The parent method must be marked `virtual`, `abstract` or `override`.

Which statements are correct?

a. I only
b. I and II only
c. II and III only
d. I, II and III
e. None of the above

Q18

A `Utils` class contains `FormatDate()`, `CalculateTax()` and `SendEmail()`, which have nothing in common. This is an example of:

a. Coincidental cohesion
b. Logical cohesion
c. Temporal cohesion
d. Functional cohesion
e. Sequential cohesion

Q19

`GoldTraveller` inherits from `Traveller`. What happens with the code below?

```
List<GoldTraveller> golds = new List<GoldTraveller>();
List<Traveller> travellers = golds;
```

a. It compiles, because a `GoldTraveller` is a `Traveller`.
b. It compiles, but throws `InvalidCastException` at run time.
c. It compiles, and `travellers` becomes a copy of `golds`.
d. Compile-time error: a `List<GoldTraveller>` is not a `List<Traveller>`, even though a `GoldTraveller` is a `Traveller`.
e. None of the above.

Q20

How should the class below be changed to follow the Dependency Inversion Principle?

```
class OrderService {
    private SqlDatabase db = new SqlDatabase();
}
```

a. Make `SqlDatabase` a static class.
b. Make `OrderService` inherit from `SqlDatabase`.
c. Depend on an `IDatabase` interface and receive it through the constructor.
d. Copy the database code into `OrderService`.
e. Make the `db` field public.

Q21

`TravellerController.Book()` checks the traveller's points, subtracts the cost, and prints "Booked!" with `Console.WriteLine`, while `Traveller` only has `{ get; set; }` properties. Which statement is the most accurate?

a. This is the recommended "fat model, thin controller" design.
b. The controller is a good example of the Interface Segregation Principle.
c. This is a fat controller with an anemic model: the controller does the Model's job and the View's job.
d. This follows MVC because all the code is in one place.
e. None of the above.

Q22

Income is tax-free up to and including $18200. Which pair of inputs best tests the **boundary** of this rule?

a. 100 and 20000
b. -1 and 1000000
c. 5000 and 10000
d. 18200 and 18201
e. 0 and 100

Q23

A `DiscountCalculator` has a `switch` on the customer type that must be edited every time a new customer type is added. What is the best way to fix it?

a. Add a comment reminding developers to update the switch.
b. Replace the switch with a chain of if-else statements.
c. Move the switch into the Main method.
d. Make the customer type a public field so any class can change it.
e. Give each customer type its own class that overrides a virtual `GetDiscount()` method.

Q24

Class `A` changes a private field of class `B` directly using reflection. Which type of coupling is this?

a. Data coupling
b. Stamp coupling
c. Control coupling
d. Content coupling
e. Common coupling

Q25

A `Startup` class contains `InitLogger()`, `LoadConfig()` and `OpenDatabase()`. They are grouped only because they all run when the program starts. This is an example of:

a. Coincidental cohesion
b. Logical cohesion
c. Temporal cohesion
d. Communicational cohesion
e. Functional cohesion

Q26

A `DiscountCalculator` has a `switch` on the customer type ("Regular", "Silver", "Gold"). Every time a new customer type is added, a developer must edit this `switch`. Which SOLID principle is violated?

a. Single Responsibility Principle.
b. Open-Closed Principle.
c. Liskov Substitution Principle.
d. Interface Segregation Principle.
e. Dependency Inversion Principle.

Q27

Consider these statements:

I. Overloading is resolved at compile time.
II. Overriding is resolved at run time.
III. Overriding happens between two methods in the same class.
IV. Overloading requires the `virtual` keyword.

Which statements are correct?

a. I and II only
b. III and IV only
c. I, II and III only
d. All of them
e. None of them

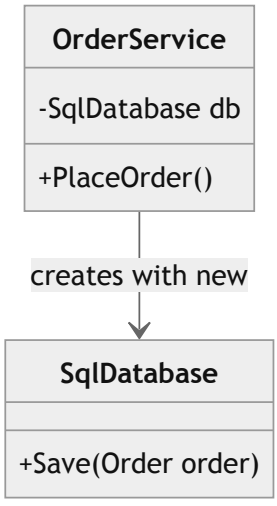
Q28

A method's contract says nothing about what happens when the input is negative. Should the test plan include a test with input `-1`?

a. Yes, every test plan must include negative input.
b. Yes, and the expected outcome is an `ArgumentException`.
c. No, because if the contract does not say what happens with invalid input, you cannot test with invalid input.
d. No, because negative numbers cannot be passed to methods in C#.
e. Yes, and the expected outcome is 0.

Q29

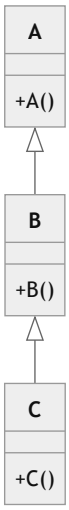
In the design below, `OrderService` creates its own `SqlDatabase` with `new SqlDatabase()`. Which SOLID principle does this violate most directly?



a. Liskov Substitution Principle.
b. Interface Segregation Principle.
c. Open-Closed Principle.
d. Single Responsibility Principle.
e. Dependency Inversion Principle.

Q30

Each constructor below writes its own class name with `Console.Write`. What does `new C();` print?



a. C
b. CBA
c. ABC
d. AC
e. Compile-time error.

Q31

What does the code below print?

```
class Animal { }
class Dog : Animal { }

void Speak(Animal a) {
    Console.Write("Animal ");
}

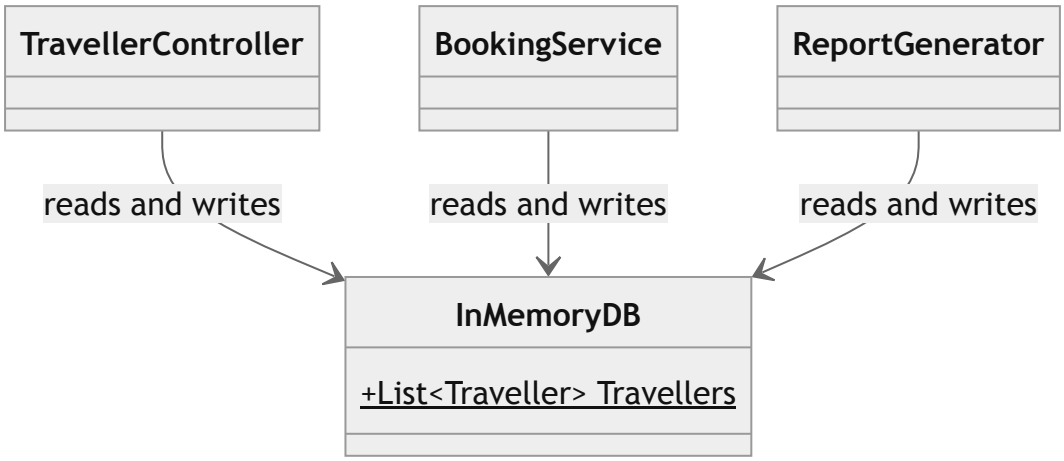
void Speak(Dog d) {
    Console.Write("Dog ");
}

Animal pet = new Dog();
Speak(pet);
Speak(new Dog());
```

a. `Dog Dog`
b. `Animal Dog`
c. `Animal Animal`
d. `Dog Animal`
e. Compile-time error, because the call is ambiguous.

Q32

In the design below, several classes read and write `InMemoryDB.Travellers`, a `public static` list. Which type of coupling is this?



a. Data coupling
b. Common coupling
c. Stamp coupling
d. Control coupling
e. Content coupling

Q33

In MSTest, which attribute marks a method that must run **before every test** in the class?

a. `[TestClass]`
b. `[TestMethod]`
c. `[DataRow]`
d. `[TestInitialize]`
e. `[Timeout]`

Q34

What happens when the code below is compiled?

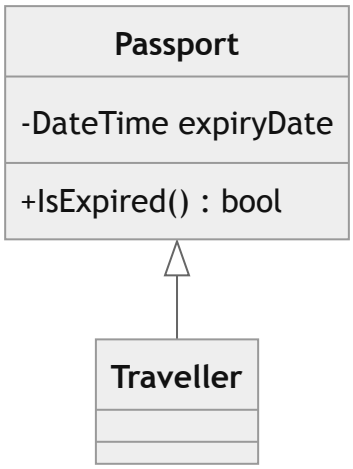
```
class Person {
    public Person(string name) { }
}

class Student : Person {
    public Student() { }
}
```

a. It compiles, and Person's constructor is skipped.
b. It compiles, and name is set to null.
c. It compiles, but throws an exception at run time.
d. Compile-time error, because Person has no parameterless constructor and Student does not call base(...).
e. None of the above.

Q35

A developer designs the classes below so that `Traveller` can reuse `IsExpired()`. What is the better design?



a. Make `Passport` inherit from `Traveller`.
b. Make `IsExpired()` static.
c. Copy `IsExpired()` into `Traveller`.
d. Make `Passport` an interface with all of Traveller's methods.
e. Use composition: `Traveller` has a private `Passport` field and calls `passport.IsExpired()`.

Q36

A class has `ReadFile()`, `ParseLines()` and `SaveRecords()`, where the output of each method is the input of the next, like an assembly line. This is an example of:

a. Procedural cohesion
b. Communicational cohesion
c. Temporal cohesion
d. Sequential cohesion
e. Coincidental cohesion

Q37

A `ReportGenerator` class loads sales data from the database, formats it as HTML, and emails it to the manager. Which description fits best?

a. Low cohesion and a violation of SRP; it should be split into separate classes for loading, formatting and sending.
b. High cohesion, and it follows SRP.
c. Loose coupling, and it follows ISP.
d. Functional cohesion, and it follows OCP.
e. None of the above.

Q38

`Run()` is called. What is printed?

```
void Run() {
    try { MethodA(); }
    catch (Exception) { Console.Write("Run "); }
}

void MethodA() {
    try { MethodB(); }
    catch (Exception) { Console.Write("A "); }
}

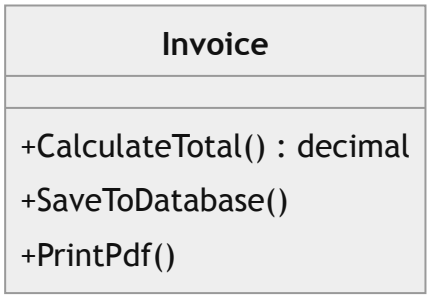
void MethodB() {
    MethodC();
    Console.Write("B ");
}

void MethodC() {
    throw new Exception();
}
```

a. `Run`
b. `A`
c. `B A`
d. `A Run`
e. Nothing, the program terminates.

Q39

Which SOLID principle does the `Invoice` class below violate?



a. Single Responsibility Principle.
b. Open-Closed Principle.
c. Liskov Substitution Principle.
d. Interface Segregation Principle.
e. Dependency Inversion Principle.

Q40

What does `new B();` print?

```
class A {
    public A() {
        Show();
    }
    public virtual void Show() {
        Console.Write("A");
    }
}

class B : A {
    private string name = "B";

    public B() {
        name = "C";
    }
    public override void Show() {
        Console.Write(name);
    }
}
```

a. A
b. B
c. C
d. Nothing, because name is still null.
e. Compile-time error.
